

SOFTWAREDESIGN EN -KWALITEIT

LOOSE COUPLING
EN HET
FACTORY PATTERN

avans
hogeschool

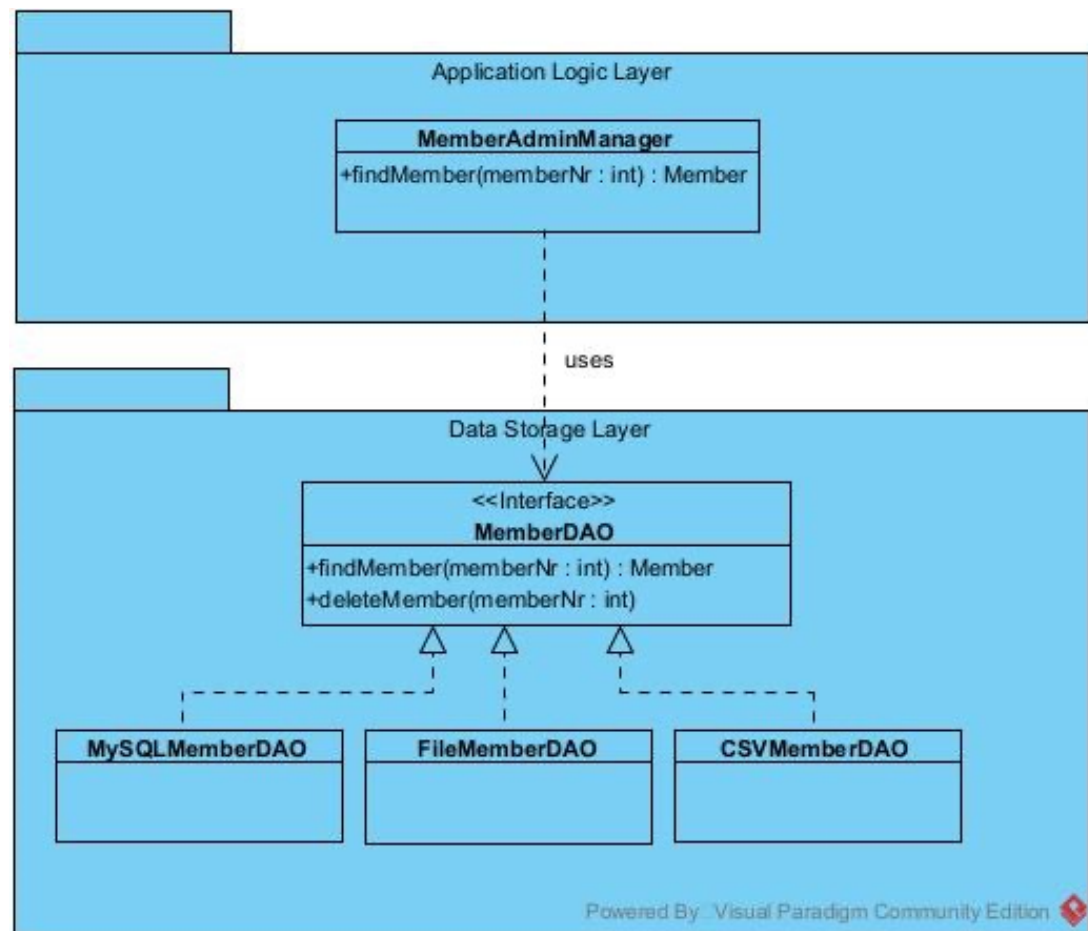


NA DEZE LES ...

1. Kun je dependency inversion toepassen
2. Weet je hoe je loose coupling kunt verkrijgen
3. Kun je uitwisselbare modules maken in je software

HUISWERK: GEBRUIK VAN DAO'S

- Aandachtspunt is de relatie tussen klassen in de Application Logic laag en de Data Storage laag.
- Discussie: uitwerkingen

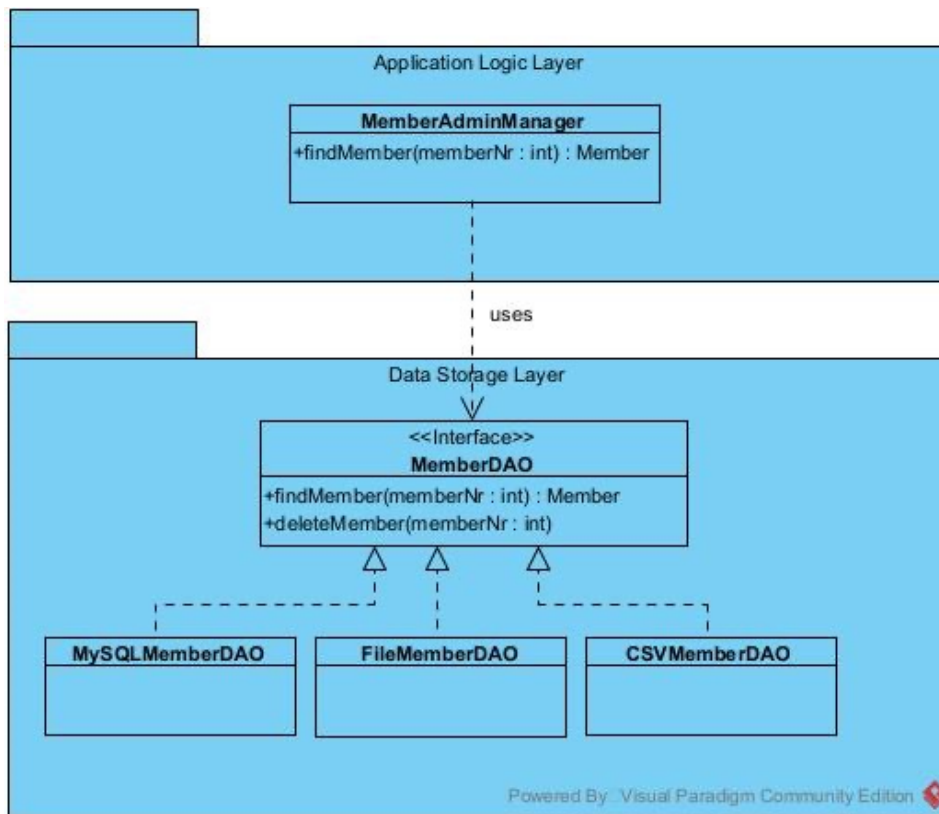


UITWERKING HUISWERK

Voorbeeld uitwerking als demonstratie

Gebruik van refactoring tools zoals 'Extract Interface'

HET GEBRUIK VAN DAO'S IN CODE



```
public Member findMember(int memberNr)
```

```
{
```

```
    Member member = null;
```

```
    // find member in ArrayList attribute
```

```
    // if not found, try a datasource
```

```
    MemberDAO memberDAO = new MySQLMemberDAO();
```

```
    member = memberDAO.findMember(memberNr);
```

```
    // ...
```

```
}
```

Nee!

Is dit program to an interface?
Is dit loose coupling?

DEPENDENCY INJECTION

```
public class MemberAdminManager {  
  
    private MemberDAO memberDAO;  
  
    public MemberAdminManager(MemberDAO memberDAO) {  
        this.memberDAO = memberDAO;  
    }  
  
    ...  
}
```

Is dit loose- of tight/strong coupling?

WAT DOEN WE MET DE ANDERE DAO'S

```
public class MemberAdminManager {  
  
    private MemberDAO memberDAO;  
    private BookDAO bookDAO;  
    private CopyDAO copyDAO;  
    private LoanDAO loanDAO;  
    private ReservationDAO reservationDAO;  
  
    public MemberAdminManager(MemberDAO memberDAO,  
                               BookDAO bookDAO,  
                               CopyDAO copyDAO,  
                               LoanDAO loanDAO,  
                               ReservationDAO reservationDAO) {  
  
        this.memberDAO = memberDAO;  
        this.bookDAO = bookDAO;  
        this.copyDAO = copyDAO;  
        this.loanDAO = loanDAO;  
        this.reservationDAO = reservationDAO;  
    }  
}
```

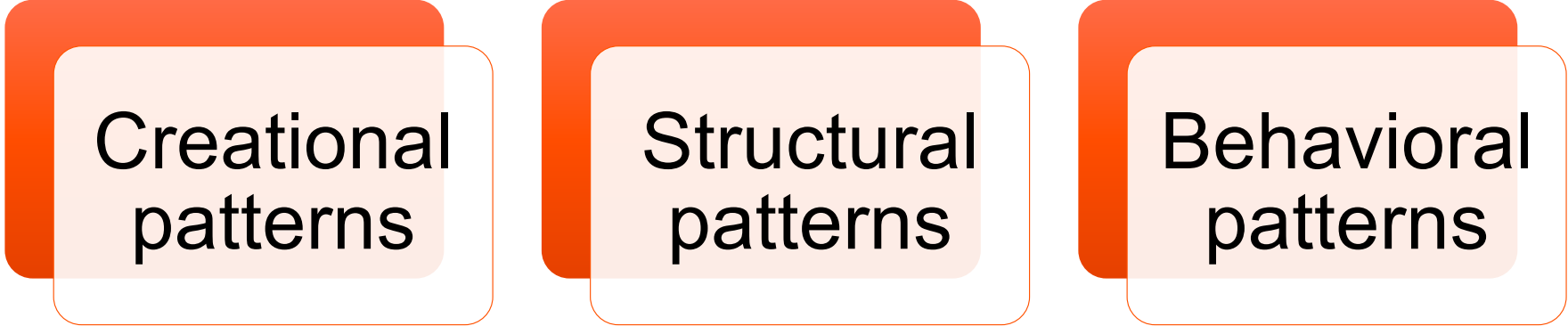


...

}

Hoe zit het met de overzichtelijkheid?
Denk daarbij ook aan de code die de MemberAdminManager instantieert!

DESIGN PATTERNS



Creational
patterns

Structural
patterns

Behavioral
patterns

In LU2 komen we nog uitgebreid terug op designpatterns

HET FACTORY PATTERN

- ☐ Met het creëren van objecten van een klasse (`new`) heb je per definitie strong coupling
- ☐ Je moet immers de concrete klasse aangeven
- ☐ We kunnen dit niet voorkomen
(logisch: anders hadden we nooit objecten)
- ☐ Oplossing is dus om het aantal plaatsen waar objecten gecreëerd wordt te beperken
- ☐ Doe dit niet door de hele code, maar daar waar je loose coupling wilt omdat daar variatie te verwachten is en je te veel wijzigingen wilt voorkomen

SIMPLE FACTORY

Op 1 plek aanmaken van nieuwe objecten

```
public class SimplePizzaFactory {  
    public Pizza createPizza(String type) {  
        Pizza pizza = null;  
        if (type.equals("cheese")) {  
            pizza = new CheesePizza();  
        } else if (type.equals("pepperoni")) {  
            pizza = new PepperoniPizza();  
        } else if (type.equals("clam")) {  
            pizza = new ClamPizza();  
        } else if (type.equals("veggie")) {  
            pizza = new VeggiePizza();  
        }  
        return pizza;  
    }  
}
```

Bron: <https://stackoverflow.com/questions/56568317/what-are-the-disadvantages-of-simple-factory-compared-to-factory-method-pattern>

SIMPLE FACTORY: *OP 1 PLEK AANMAKEN VAN NIEUWE OBJECTEN*

```
public class DAOSimpleFactory {  
  
    private final DALSource source;  
  
    public DAOSimpleFactory(DALSource source) {  
        this.source = source;  
    }  
  
    public MemberDAO createMemberDAO() {  
        switch (source) {  
            case CSV-> {  
                return new CSVMemberDAO();  
            }  
            case SQL-> {  
                return new SQLMemberDAO();  
            }  
            default->  
                throw new IllegalStateException("Unknown source " + source);  
        }  
    }  
  
    public BookDAO3 createBookDAO() {  
        switch (source) {  
            case CSV-> {  
                return new CSVBookDAO();  
            }  
            case SQL-> {  
                return new SQLBookDAO();  
            }  
            default->  
                throw new IllegalStateException("Unknown source " + source);  
        }  
    }  
}
```



Één factory om alle objecten bij een bepaalde type datasource aan te maken (op basis van parameter)



Create MemberDAO op basis van parameter



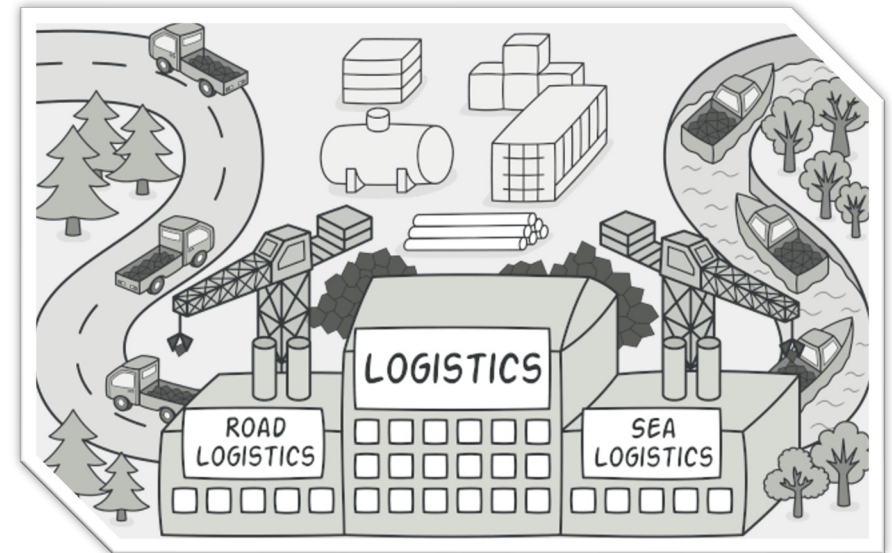
Create BookDAO op basis van parameter

HET FACTORY PATTERN

- ☐ Een factory is verantwoordelijk voor object-creatie (creational pattern)
- ☐ Een factory schermt af om welke concrete klasse het gaat
- ☐ Met een factory kun je loose coupling realiseren voor andere klassen

Een factory retourneert de gecreëerde objecten als interfaces

- ☐ Een factory zelf is strongly coupled



Bron: <https://refactoring.guru/design-patterns/factory-method>

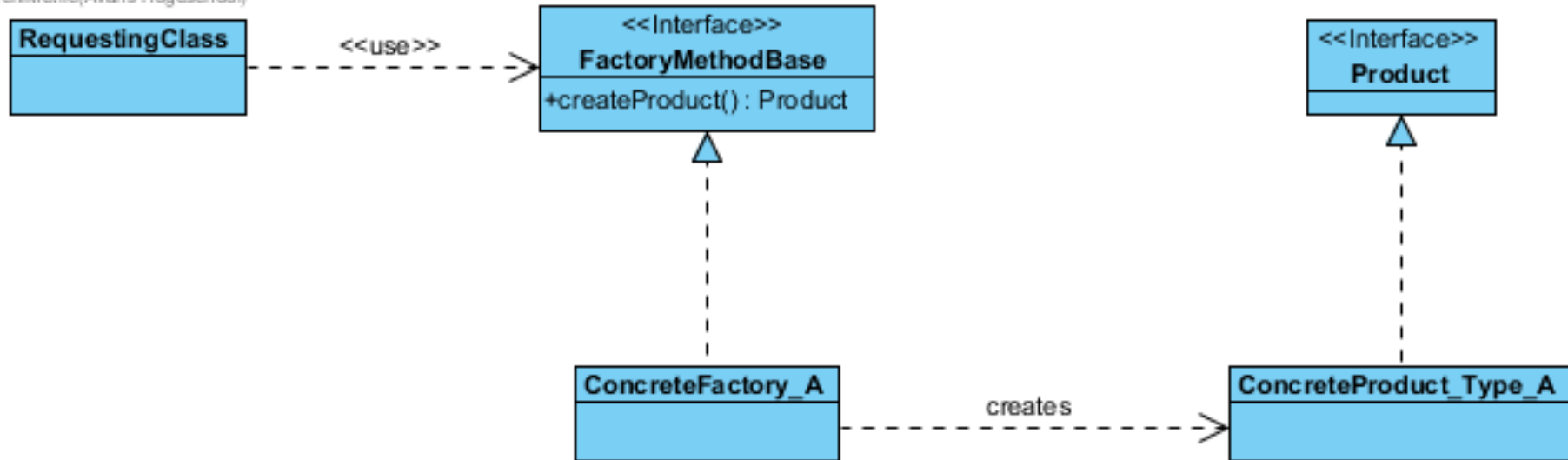
SIMPLE FACTORY: *OP 1 PLEK AANMAKEN VAN NIEUWE OBJECTEN*

```
public class DAOSimpleFactory {  
  
    private final DALSource source;  
  
    public DAOSimpleFactory(DALSource source) {  
        this.source = source;  
    }  
  
    public MemberDAO createMemberDAO() {  
        switch (source) {  
            case CSV-> {  
                return new CSVMemberDAO();  
            }  
            case SQL-> {  
                return new SQLMemberDAO();  
            }  
            default->  
                throw new IllegalStateException("Unknown source " + source);  
        }  
    }  
  
    public BookDAO3 createBookDAO() {  
        switch (source) {  
            case CSV-> {  
                return new CSVBookDAO();  
            }  
            case SQL-> {  
                return new SQLBookDAO();  
            }  
            default->  
                throw new IllegalStateException("Unknown source " + source);  
        }  
    }  
}
```

Wat als er nu een databron bijkomt?
Voldoet deze factory aan het open/closed principe?

ABSTRACT FACTORY: STAP 1

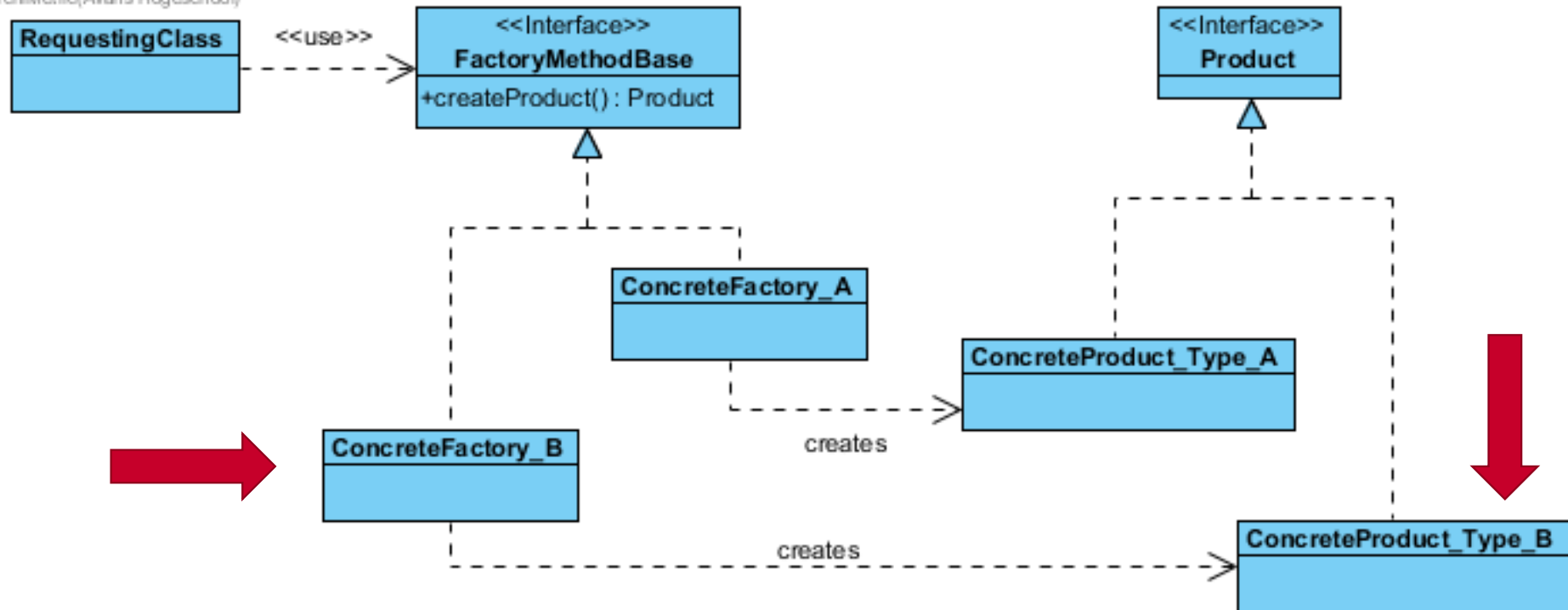
Architectuur (Avans Hogeschool)



- ❑ De klasse die een object nodig heeft (`RequestingClass`) vraagt de factory klasse het te creëren en is zo alleen op interface niveau afhankelijk
- ❑ `ConcreteFactory` creëert objecten
- ❑ `ConcreteFactory` retourneert ze als interface (`Product`)

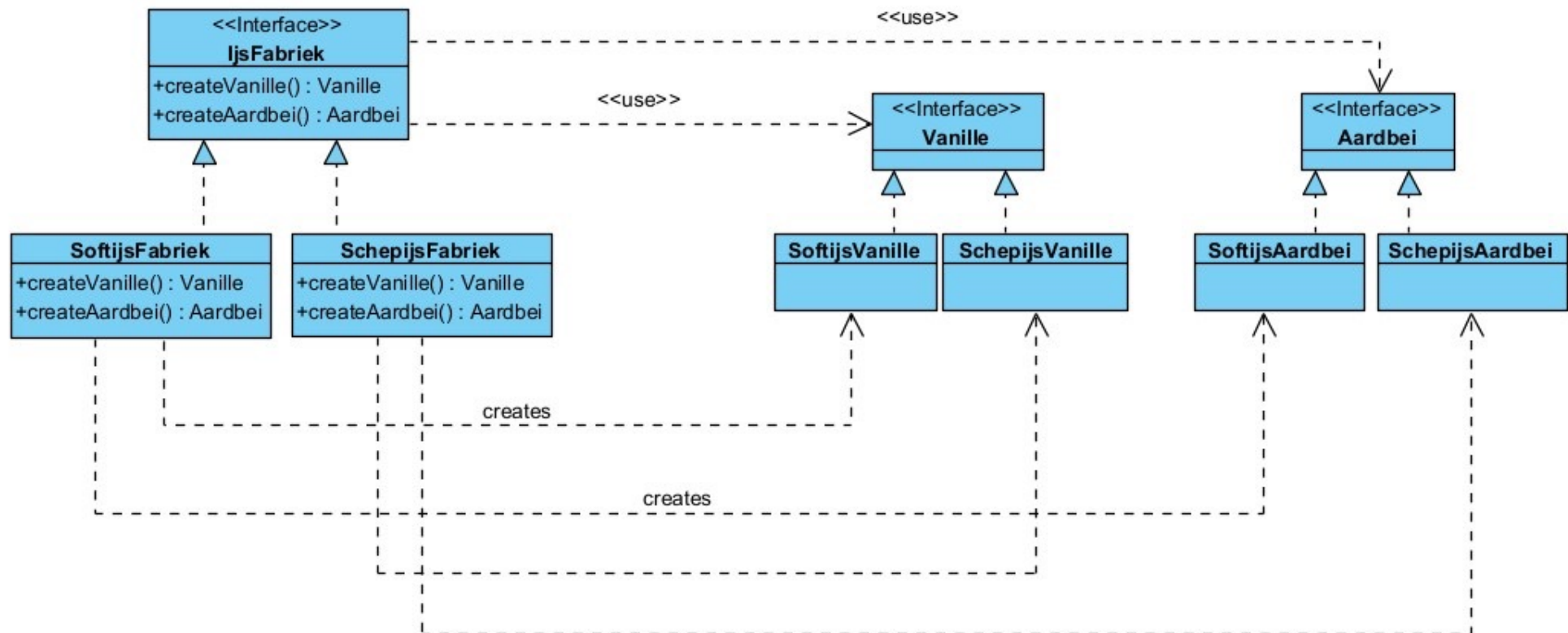
ABSTRACT FACTORY: *STAP 2*

Architectuur(Avans Hogeschool)



□ Er is een andere factory bijgekomen voor de andere soorten objecten

ABSTRACT FACTORY: IJSFABRIEK



OPDRACHT: ABSTRACT FACTORY VOOR DE BIBLIOTHEEK

☐ Pas stap 1 van het factory pattern toe op het ontwerp van de bibliotheek (UML)

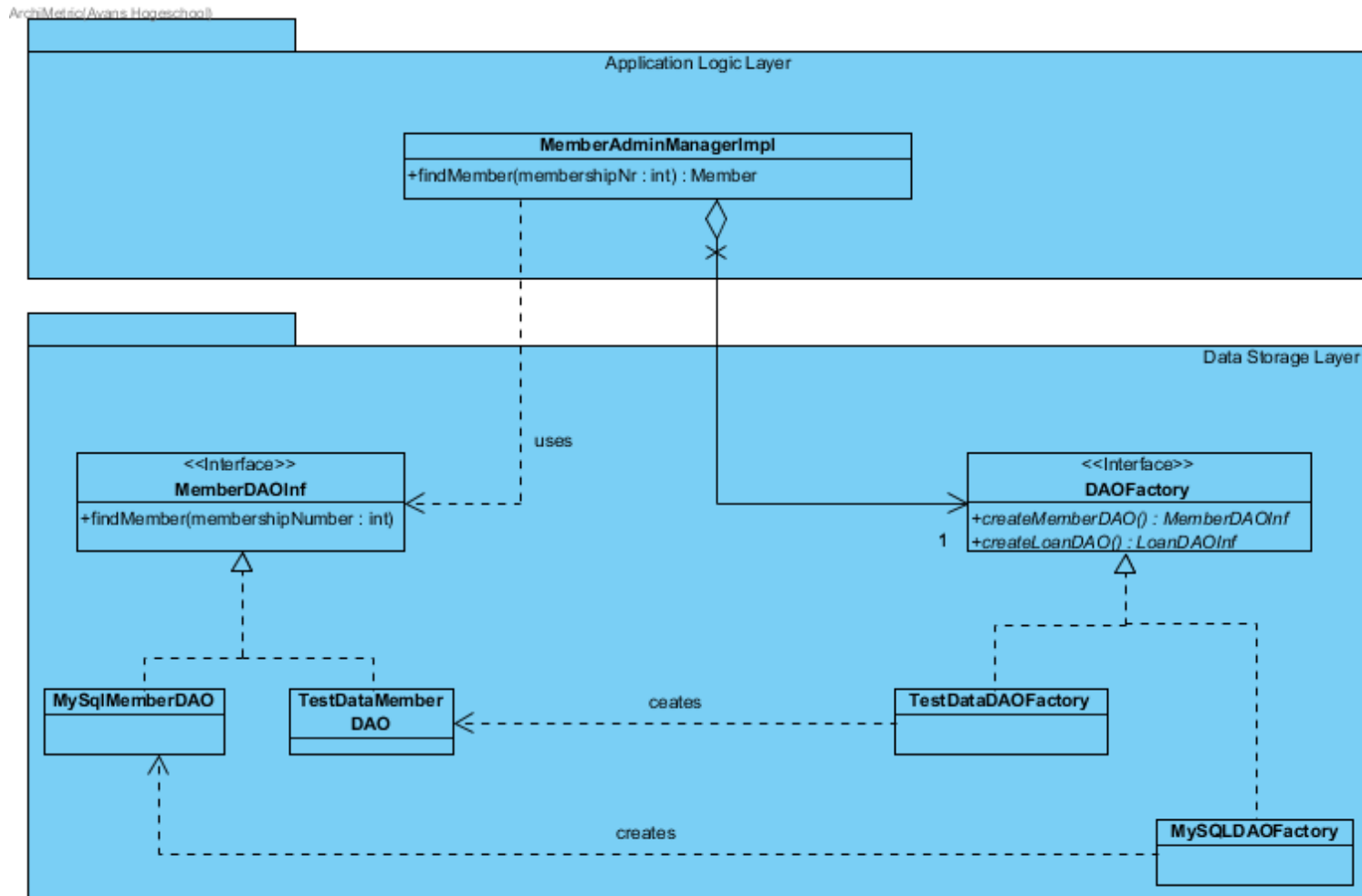
☐ Richt je op de creatie van één soort DAO die een member (*smaak*) ophaalt uit een SQLServer (*soort*) database.

Let op: SQLServer DAO mag als hardcoded uitgevoerd worden.

☐ Enkel UML! Als je tijd over hebt, dan pas overgaan naar de implementatie in code.

UITWERKING: *STAP 1 ABSTRACT FACTORY*

UITWERKING: STAP 1 ABSTRACT FACTORY



OPDRACHT: ABSTRACT FACTORY VOOR DE BIBLIOTHEEK

☐ Pas stap 2 van het factory pattern toe op het ontwerp van de bibliotheek (UML)

☐ Richt je op de creatie van meerdere DAO's (*smaken*) die entiteiten ophalen uit een SQLServer (*soort*) database.

Let op: SQLServer DAO mag als hardcoded uitgevoerd worden.

☐ Enkel UML! Als je tijd over hebt, dan pas overgaan naar de implementatie in code.

UITWERKING: *STAP 2 ABSTRACT FACTORY*

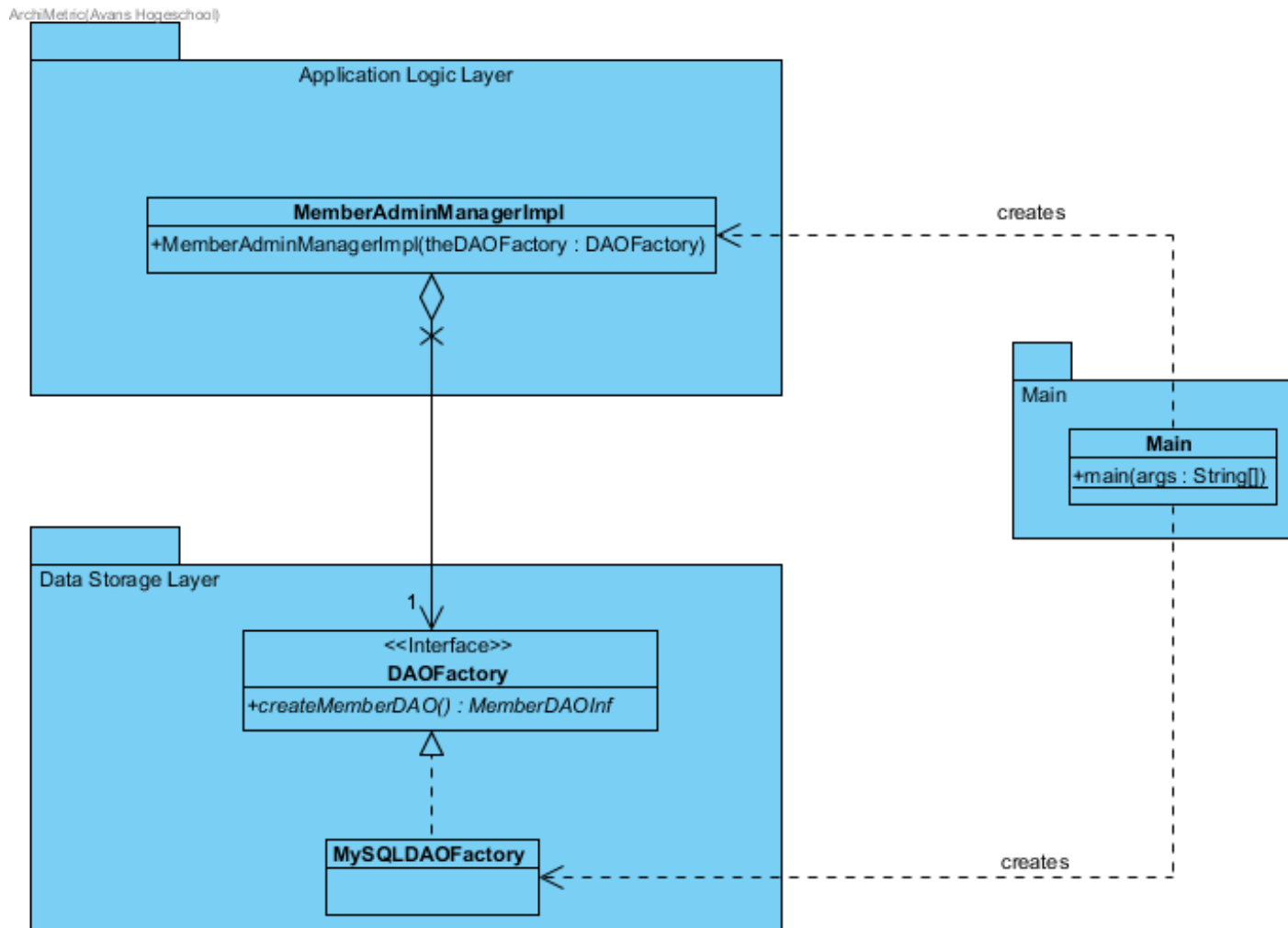
FACTORY INSTANTIATIE

- ☐ De manager klasse gebruikt de factory op interface niveau
- ☐ Maar...hoe komt de manager klasse aan een concrete factory klasse?
- ☐ Hoe kies je tussen `TestDataDAOFactory` en `SQLServerDAOFactory`?
- ☐ Een factory die een factory maakt?
En wie maakt die factory dan?

FACTORY INSTANTIATIE

- ❑ De `main` klasse instantieert een factory object
bv de `SQLServerDAOFactory`
- ❑ De `main` klasse instantieert ook de manager klasse en geeft het factory object via de constructor mee (dependency injection)
- ❑ Deze constructor heeft een parameter van type `DAOFactory` (de interface dus)
- ❑ Dit heeft een beperking:
Er is één actieve bron voor alle informatie. Want alle DAO's zijn MySQL-DAO's óf TestData-DAO's

FACTORY INSTANTIATIE



HUISWERK

Zie opdrachtomschrijving op Brightspace

Deadline: begin volgende les

avans
hogeschool



***VOLGENDE
KEER***

**Volgende keer
gaan we werken
loose coupling
en het factory
pattern**